

Contents

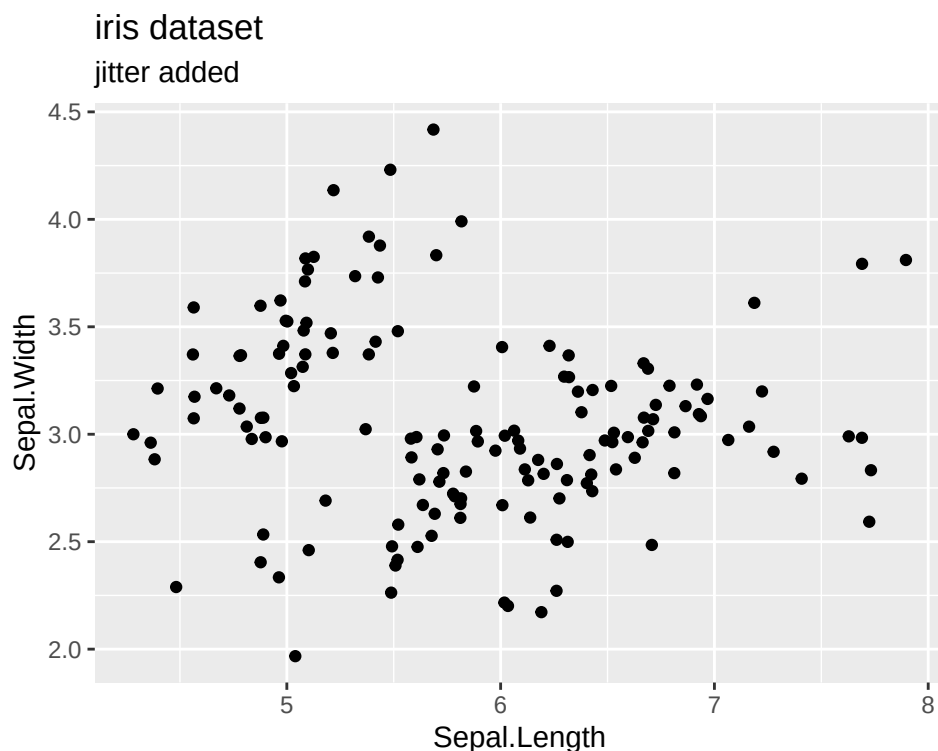
1 Clustering	2
2 k-means clustering	2
2.1 USArrests dataset	3
2.2 Extra: standardisation a.k.a. z-scoring	4
2.3 Performing k-means clustering	4
2.4 Selection of k	5
2.4.1 Extremely optional: further criteria on selection of k	6
2.5 Final k-means clustering	8
3 Hierarchical clustering	9
3.1 USArrests dataset	9
3.2 Distance matrices with <code>dist()</code>	10
3.3 Performing hierarchical clustering	10
3.4 Visualising hierarchical clustering with a dendrogram	10
3.5 Extra: getting the cluster values	11
4 Results of clustering	12
5 Conceptual Check	12

1 Clustering

Clustering is the final machine learning algorithm in this course. In contrast to the previous types of machine learning, clustering is an *unsupervised learning method*. In previous weeks, all the models we trained had a target: to *predict* an outcome variable based on some predictors.

In clustering, however, we have no such target - we aim to understand *how data may be grouped* in a dataset. For example, you may be trying to see how to group the *iris* dataset, i.e. how many distinct groups there are. While the dataset description says there are 3 species, pretend that we don't know that. How can we identify how many species there may be? How can we group the data based on just the measurements available?

For this tutorial, we will use 2 clustering algorithms, k-means clustering and hierarchical clustering, to answer this question. These clustering algorithms *rely on a notion of distance*, and so the scales of the predictors have an effect on the final outcome.



2 k-means clustering

In k-means clustering, we will specify how many clusters k there are in our dataset, and perform the following steps to determine which cluster each observation belongs to.

1. Assign each observation a random cluster $1, \dots, k$.
2. Calculate the *mean* point (centroid) of each cluster.
3. Assign each observation the cluster mean that it is closest to.

4. Repeat steps 2 and 3 until the clusters converge (do not change), or until max number of iterations is reached.

It is important to have a maximum number of iterations, as it is possible that the clustering algorithm may simply alternate between two states and never converge.

2.1 USArrests dataset

The USArrests dataset found in base R contains statistics for arrests in each U.S.A. state in 1973. We can see the rates of arrest for some crimes per 100,000 people, as well as the urban population of each state. The scales of each predictor are extremely different, and to account for that, we must *normalise* the data. Here, we will use the `scale()` function, which centers the mean to 0 and scales the standard deviation of each predictor to 1.

We will use the scaled dataset for our clustering.

```
data(USArrests)
str(USArrests)
```

```
## 'data.frame':    50 obs. of  4 variables:
## $ Murder   : num  13.2 10 8.1 8.8 9 7.9 3.3 5.9 15.4 17.4 ...
## $ Assault  : int  236 263 294 190 276 204 110 238 335 211 ...
## $ UrbanPop : int  58 48 80 50 91 78 77 72 80 60 ...
## $ Rape     : num  21.2 44.5 31 19.5 40.6 38.7 11.1 15.8 31.9 25.8 ...
```

```
summary(USArrests)
```

	Murder	Assault	UrbanPop	Rape
## Min.	: 0.800	Min. : 45.0	Min. :32.00	Min. : 7.30
## 1st Qu.:	4.075	1st Qu.:109.0	1st Qu.:54.50	1st Qu.:15.07
## Median :	7.250	Median :159.0	Median :66.00	Median :20.10
## Mean :	7.788	Mean :170.8	Mean :65.54	Mean :21.23
## 3rd Qu.:	11.250	3rd Qu.:249.0	3rd Qu.:77.75	3rd Qu.:26.18
## Max.	:17.400	Max. :337.0	Max. :91.00	Max. :46.00

```
arrests.nor <- scale(USArrests)
summary(arrests.nor)
```

	Murder	Assault	UrbanPop	Rape
## Min.	:-1.6044	Min. :-1.5090	Min. :-2.31714	Min. :-1.4874
## 1st Qu.:	-0.8525	1st Qu.:-0.7411	1st Qu.:-0.76271	1st Qu.:-0.6574
## Median :	-0.1235	Median :-0.1411	Median : 0.03178	Median :-0.1209
## Mean :	0.00000	Mean : 0.00000	Mean : 0.000000	Mean : 0.00000
## 3rd Qu.:	0.7949	3rd Qu.: 0.9388	3rd Qu.: 0.84354	3rd Qu.: 0.5277
## Max.	: 2.2069	Max. : 1.9948	Max. : 1.75892	Max. : 2.6444

2.2 Extra: standardisation a.k.a. z-scoring

This scaling and shifting of a dataset to have a mean of 0 and standard deviation of 1 is also known as *standardisation* or *z-scoring*, named after the standard normal variable with mean 0 and variance 1, $N(0, 1)$, also commonly referred to as Z . This is just another way of normalising the dataset to make its features comparable. The `scale()` function applies this transformation, which is defined as:

$$x_{\text{scaled}} = \frac{x - \text{mean}(x)}{\text{sd}(x)}$$

2.3 Performing k-means clustering

In R, we will use the `kmeans()` function in base R to perform k-means clustering. In addition to the dataset, we **must** specify the number of clusters k through the `centers` parameter. Optionally, we can also specify the number of initial states `nstart` and maximum number of iterations `iter.max`.

Once the k-means clustering algorithm converges or it reaches the maximum number of iterations, it returns the resulting clusters, as well as some statistics on the clusters, namely the *within-cluster sum of squares* and *between-cluster sum of squares*. The definition of each is not important to the course, but roughly:

- the *within-cluster sum of squares* of **each cluster** describes how spread the observations in each cluster are (compared to the cluster center);
- the *between-cluster sum of squares* describes how spread the **cluster centers** are from each other.

```
set.seed(1337)
kmeans.2 <- kmeans(arrests.nor, centers = 2, nstart = 25, iter.max = 10)
kmeans.2
```

```
## K-means clustering with 2 clusters of sizes 20, 30
```

```
##
```

```
## Cluster means:
```

```
##      Murder      Assault      UrbanPop      Rape
```

```
## 1  1.004934  1.0138274  0.1975853  0.8469650
```

```
## 2 -0.669956 -0.6758849 -0.1317235 -0.5646433
```

```
##
```

```
## Clustering vector:
```

```
##      Alabama      Alaska      Arizona      Arkansas      California
```

```
##           1           1           1           2           1
```

```
##      Colorado      Connecticut      Delaware      Florida      Georgia
```

```
##           1           2           2           1           1
```

```
##      Hawaii      Idaho      Illinois      Indiana      Iowa
```

```
##           2           2           1           2           2
```

```
##           Kansas      Kentucky      Louisiana      Maine      Maryland
##           2           2           1           2           1
## Massachusetts      Michigan      Minnesota      Mississippi      Missouri
##           2           1           2           1           1
##           Montana      Nebraska      Nevada      New Hampshire      New Jersey
##           2           2           1           2           2
##           New Mexico      New York      North Carolina      North Dakota      Ohio
##           1           1           1           2           2
##           Oklahoma      Oregon      Pennsylvania      Rhode Island      South Carolina
##           2           2           2           2           1
##           South Dakota      Tennessee      Texas      Utah      Vermont
##           2           1           1           2           2
##           Virginia      Washington      West Virginia      Wisconsin      Wyoming
##           2           2           2           2           2
##
## Within cluster sum of squares by cluster:
## [1] 46.74796 56.11445
## (between_SS / total_SS = 47.5 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

2.4 Selection of k

The above was done for only a single value of k - and so we will have to test *multiple* values of k . For each value of k , we will consider the *total within-cluster sum-of-squares* as a measure of the variance within each cluster. As the number of clusters increases, the within-cluster variation naturally decreases (at the limit, when number of clusters = number of observations, the within-cluster variation is 0, since each cluster only has 1 point).

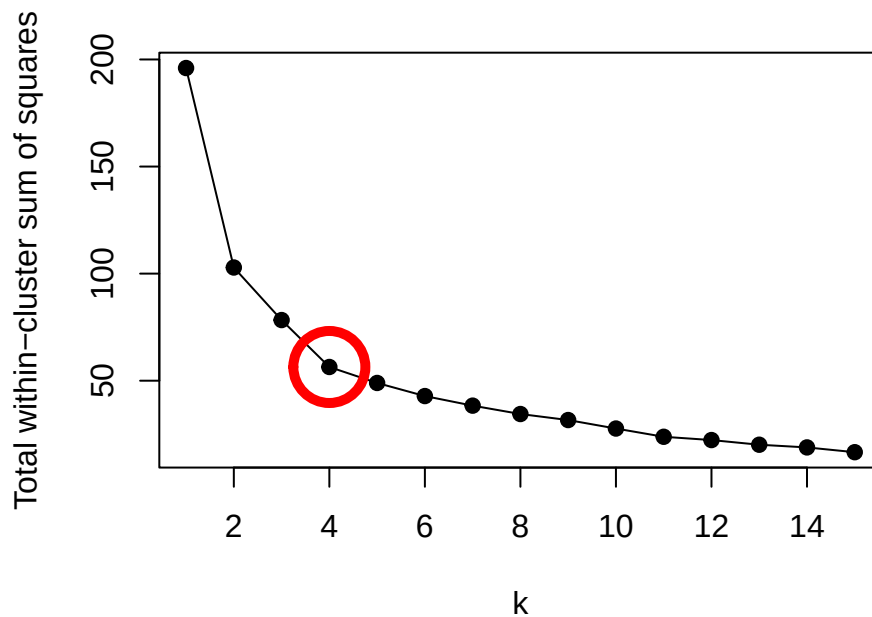
So, to choose an appropriate value of k , we will use the ‘elbow method’, in which we choose the value of k that has a ‘bend’ in the plotted curve. This corresponds to a “decreasing rate of return” of adding one more cluster. The ‘bend’ occurs at $k = 4$, so we use that as the final number of clusters for the dataset.

```
wcss <- function(k) {
  set.seed(100)
  kmeans(arrests.nor, k, nstart = 10)$tot.withinss
}

wcss_k <- sapply(1:15, wcss)
```

```
k = 4
```

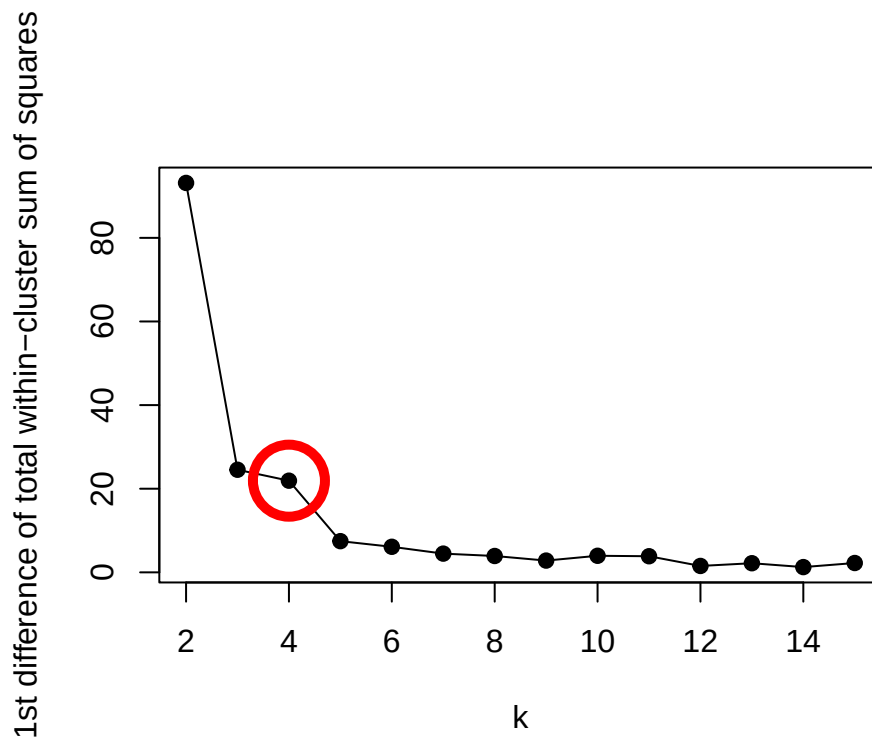
```
plot(wcss_k, type="o", pch=19, xlab="k", ylab="Total within-cluster sum of  
↪ squares")  
points(x=k, y=wcss_k[k], pch=1, cex=5, col="red", lwd=5)
```



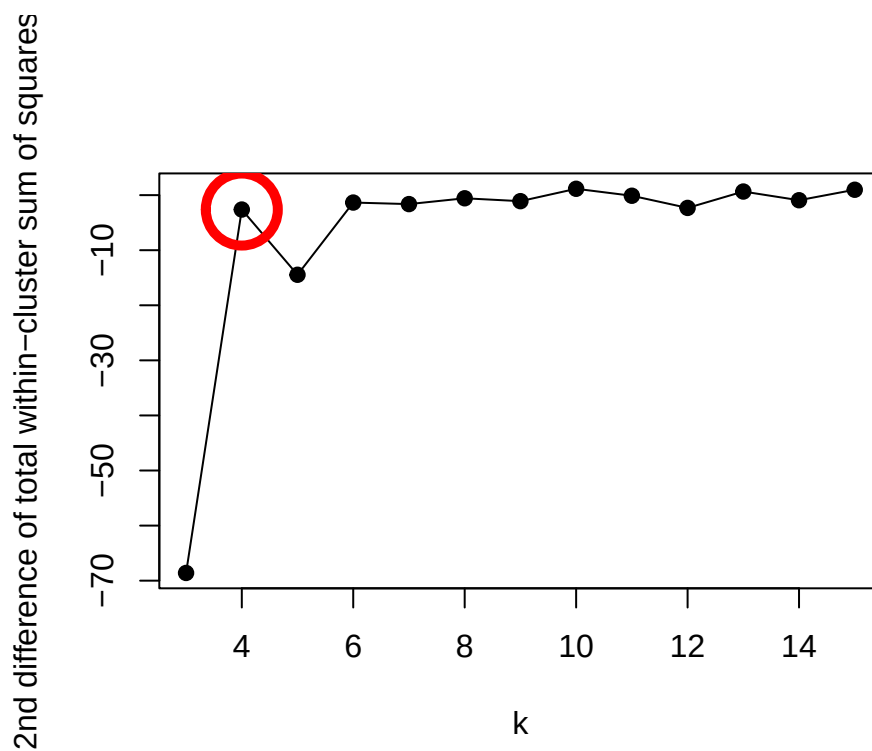
2.4.1 Extremely optional: further criteria on selection of k

We may also consider the differences between the total within-cluster sum of squares, which can be calculated with `diff()`. The second difference shows where a good choice of k is, indicated by a sudden spike and then decrease. The theory for this is not complicated, but does take some thinking through.

```
plot(2:15, -diff(wcss_k), type="o", pch=19, xlab="k", ylab="1st difference  
↪ of total within-cluster sum of squares")  
points(x=k, y=-diff(wcss_k)[k-1], pch=1, cex=5, col="red", lwd=5)
```



```
plot(3:15, -diff(wcss_k, differences=2), type="o", pch=19, xlab="k",
     ↪ ylab="2nd difference of total within-cluster sum of squares")
points(x=k, y=-diff(wcss_k, differences=2)[k-2], pch=1, cex=5, col="red",
     ↪ lwd=5)
```



2.5 Final k-means clustering

Looking at the final clustering result with $k = 4$, we can see the *scaled* cluster centers as well as to which cluster each state belongs to. The ordering of the clusters (the cluster numbers) may be different, though the states are clustered the same. The scaled centers do not represent any real-world statistic, so to truly understand what these clusters represent, we have to look at the *unscaled* cluster centers.

```
set.seed(100)
```

```
kmeans.final <- kmeans(arrests.nor, 4)
```

```
kmeans.final
```

```
## K-means clustering with 4 clusters of sizes 8, 16, 13, 13
```

```
##
```

```
## Cluster means:
```

```
##      Murder    Assault  UrbanPop      Rape
## 1  1.4118898  0.8743346 -0.8145211  0.01927104
## 2 -0.4894375 -0.3826001  0.5758298 -0.26165379
## 3 -0.9615407 -1.1066010 -0.9301069 -0.96676331
## 4  0.6950701  1.0394414  0.7226370  1.27693964
##
```

```
## Clustering vector:
```

```
##      Alabama      Alaska      Arizona      Arkansas      California
##           1           4           4           1           4
##      Colorado  Connecticut  Delaware      Florida      Georgia
##           4           2           2           4           1
##      Hawaii      Idaho      Illinois      Indiana      Iowa
##           2           3           4           2           3
##      Kansas      Kentucky  Louisiana      Maine      Maryland
##           2           3           1           3           4
##      Massachusetts  Michigan  Minnesota  Mississippi  Missouri
##           2           4           3           1           4
##      Montana      Nebraska      Nevada  New Hampshire  New Jersey
##           3           3           4           3           2
##      New Mexico      New York  North Carolina  North Dakota      Ohio
##           4           4           1           3           2
##      Oklahoma      Oregon  Pennsylvania  Rhode Island  South Carolina
##           2           2           2           2           1
##      South Dakota      Tennessee      Texas      Utah      Vermont
##           3           1           4           2           3
##      Virginia      Washington  West Virginia  Wisconsin      Wyoming
##           2           2           3           3           2
##
```

```
## Within cluster sum of squares by cluster:
```



```
## [1] 8.316061 16.212213 11.952463 19.922437
## (between_SS / total_SS = 71.2 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

The calculation of the cluster mean is achieved by simply adding the cluster information to the original, unscaled data, and calculating the cluster means from there. In the notes, `summarise_all()` is used. I chose to use `summarise()` with `across()` to allow the addition of an additional statistic, the number of states in the cluster.

In the resulting clustering, we can see that clusters 2 and 3 have a lower murder and assault rate, and differ in their urban population. We may choose to focus on these two clusters for further study in the real world.

```
USArrests %>% mutate(Cluster=kmeans.final$cluster) %>%
  group_by(Cluster) %>% summarise(across(everything(), mean), count = n())
```

```
## # A tibble: 4 x 6
##   Cluster Murder Assault UrbanPop Rape count
##   <int>   <dbl>   <dbl>   <dbl> <dbl> <int>
## 1     1    13.9    244.    53.8  21.4     8
## 2     2     5.66   139.    73.9  18.8    16
## 3     3     3.6    78.5    52.1  12.2    13
## 4     4    10.8   257.    76    33.2    13
```

3 Hierarchical clustering

Hierarchical clustering is more of a *bottom-up* approach, and is described as an agglomerative clustering method, where we take every observation to be a cluster by itself, and procedurally combine the closest clusters, until the whole dataset is a single cluster. The method by which the clusters are combined may be defined differently: for example, we may choose to combine the 2 clusters whose means are closest together. Alternatively, we may also choose to measure the minimum or maximum distance between each cluster (based on the observations in each cluster) and combine the 2 clusters with the smallest minimum or maximum distance.

3.1 USArrests dataset

Again, we will see how hierarchical clustering performs on the USArrests dataset. To perform a hierarchical clustering, we use the `hclust()` function in base R, which requires a *distance matrix*, computed with the `dist()` function in base R.

3.2 Distance matrices with `dist()`

A **distance matrix** is a square matrix that describes the *distance* from every single point to every other. If we have n observations $x_1, \dots, x_n$, then the (i, j) th entry in the distance matrix describes how far x_i is from x_j . Distance is symmetric (the distance from x_i to x_j is the same as the distance from x_j to x_i), so the distance matrix is also symmetric about the diagonal.

Also, there are multiple ways to calculate a distance - again, we will rely on the *Euclidean measure*, which is the natural notion of distance.

3.3 Performing hierarchical clustering

We first compute the distance matrix with `dist()`. The default argument for the `method` parameter is "euclidean". Once the distance matrix is computed, we can pass it in to `hclust()`, and optionally specify the method of how the clusters are combined. By default, `method="complete"`.

With `method="complete"`, the two clusters with the smallest maximum distance between points in the cluster are combined, "average" combines the closest cluster centers, and "single" combines the two clusters with the smallest minimum distance between points in the cluster.

```
distance <- dist(arrests.nor, method = "euclidean")
h.clust <- hclust(distance, method = "complete")
```

3.4 Visualising hierarchical clustering with a dendrogram

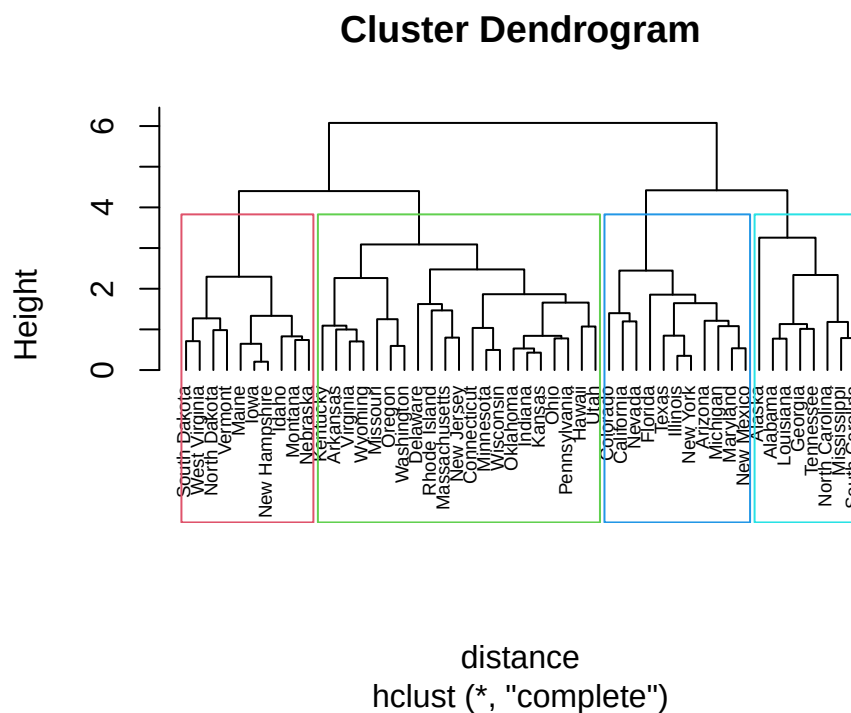
Once the clustering is complete, we can visualise the results with a **dendrogram**. The height on the y-axis indicates the distance at which each cluster was joined. If we draw a horizontal line at any level, the number of times it intersects corresponds to the number of clusters at that level. For example, drawing a line at $y = 4$ intersects 4 lines, so there were 4 clusters at that point.

To determine a suitable number of clusters, we can see the point at which the jump in height significantly increases - in this case, when the number of clusters decreases from 4 to 3 (and then quickly to 2) clusters.

We can draw a rectangle around the clusters in the dendrogram with `rect.hclust()`, specifying the number of clusters k , and optionally the colours to use with `border`. In this case, $k = 4$, and we choose four colours with `border = 2:5`.

```
# cex - text size, hang - position of labels
plot(h.clust, cex = 0.6, hang = -1)

# k - number of clusters, border - colours to use
rect.hclust(h.clust, k = 4, border = 2:5)
```



3.5 Extra: getting the cluster values

To get the final clustering results, we can use the `cutree()` function with the output of `hclust()`, specifying how many clusters we want. You can then process the data in a similar way to get the unscaled data means.

```
h.clust.4 <- cutree(h.clust, 4)
h.clust.4
```

##	Alabama	Alaska	Arizona	Arkansas	California
##	1	1	2	3	2
##	Colorado	Connecticut	Delaware	Florida	Georgia
##	2	3	3	2	1
##	Hawaii	Idaho	Illinois	Indiana	Iowa
##	3	4	2	3	4
##	Kansas	Kentucky	Louisiana	Maine	Maryland
##	3	3	1	4	2
##	Massachusetts	Michigan	Minnesota	Mississippi	Missouri
##	3	2	3	1	3
##	Montana	Nebraska	Nevada	New Hampshire	New Jersey
##	4	4	2	4	3
##	New Mexico	New York	North Carolina	North Dakota	Ohio
##	2	2	1	4	3
##	Oklahoma	Oregon	Pennsylvania	Rhode Island	South Carolina

##	3	3	3	3	1
##	South Dakota	Tennessee	Texas	Utah	Vermont
##	4	1	2	3	4
##	Virginia	Washington	West Virginia	Wisconsin	Wyoming
##	3	3	4	3	3

4 Results of clustering

Once the clustering is complete, we can use the clustering to then further understand the data. For example, we may choose to focus on the cluster of states which have a higher rate of arrests for murder and assault, trying to identify if there may be any causal link between them. This process is separate from the data analysis.

Note that each clustering method, while largely the same, still had slight differences between them.

5 Conceptual Check

1. How does clustering differ from classification?
2. Describe briefly the algorithms for k-means and hierarchical clustering.
3. How is the number of clusters chosen in k-means and hierarchical clustering?
4. Is scaling important in k-means or hierarchical clustering? Why or why not?